دليل Git لفريق Work_Tech

إزاي تنزّل المشروع، تشتغل على البرانش بتاعك، ترفع شغلك، وتسحب آخر تحديثات. كل خطوة ومعاها الأوامر.

الريبو: `https://github.com/salemsalehtech/Work_Tech.git`
البرانش الأساسي (Default): `develop` ، وكل الشغل بيتجمع فيه.
البرانش `main`: للنسخة النهائية بس. صاحب الريبو هو اللي بينقل عليه.

الريبو القديم `nhahub/NHA-5-046` مبقاش مستخدم. متعملوش push أو pull منه. الشغل كله على `salemsalehtech/Work_Tech` بس. لو عندك نسخة من القديم، شوف القسم 2 عشان تحوّلها.

```
feature/<task>  →  git push  →  Pull Request  →  develop  →  main
```

قواعد مهمة:

- متعملش `push` على `develop` أو `main` مباشرة. اشتغل على البرانش بتاعك، وبعدها افتح Pull Request.
- اسحب آخر تحديثات من `develop` قبل ما تبدأ شغل كل يوم.
- رسالة الـ commit لازم تبقى بالشكل المطلوب (شوف القسم 5)، وإلا الـ commit هيترفض.

1. أول مرة بس: تجهيز الجهاز

محتاج **Git** و**Node.js 24** أو أحدث. اتأكد منهم:

```
git --version
node -v        # لازم يكون v24 أو أحدث
```

عرّف Git باسمك وإيميلك (نفس إيميل حساب GitHub):

```
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
git config --global pull.rebase false   # عشان git pull ميقفش برسالة divergent branches
```

2. أول مرة بس: تنزيل المشروع

```
git clone https://github.com/salemsalehtech/Work_Tech.git
cd Work_Tech
npm install
npm run dev     # تشغيل المشروع للتجربة
```

لو عندك نسخة قديمة من `nhahub/NHA-5-046` أو بأي لينك تاني، صلّح اللينك بدل ما تنزّل من جديد:

```
git remote set-url origin https://github.com/salemsalehtech/Work_Tech.git
git remote -v   # للتأكد
```

3. برانش لكل Task

كل task ليها برانش لوحدها، بتتعمل من `develop`، واسمها بالشكل ده: `feature/<scope>-<short-name>`

الـ Task	اسم البرانش
فلاتر صفحة المنتجات	`feature/catalog-filters`
صفحة الكارت	`feature/cart-page`
الـ Header والـ Footer	`feature/layout-header-footer`

تبدأ task جديدة

```
git checkout develop
git pull origin develop
git checkout -b feature/catalog-filters
git push -u origin feature/catalog-filters  # أول مرة GitHub يرفعه على
```

ترجع لـ task بدأتها قبل كده

```
git fetch origin
git checkout feature/catalog-filters
git pull
```

اعرف إنت على أنهي برانش:

```
git branch      # البرانش اللي عليه نجمة * هو اللي إنت فيه
git status
```

4. كل يوم قبل ما تبدأ: هات آخر كود

الخطوة دي بتجيبلك شغل باقي الفريق اللي اتدمج في `develop` ، وتحطه في البرانش بتاعك:

قبلها: لازم تكون حافظ شغلك. لو `git status` بيقول إن فيه تعديلات، اعمل commit (القسم 5) أو اركنها مؤقتاً:

```
git stash        # يركن التعديلات
# ... نفّذ الأوامر اللي تحت ...
git stash pop    # يرجّعها
```

```
git checkout develop
git pull origin develop
git checkout feature/your-task
git merge develop --no-edit   # --no-edit عشان ميفتحش محرر vim
npm install     # لو حد ضاف مكتبة جديدة
```

لو ظهر **conflict**، افتح الملفات اللي بيقول عليها في VS Code واختار الكود الصح. بعدها:

```
git add .
git commit -m "chore: resolve merge conflicts with develop"
```

5. حفظ شغلك (Commit)

```
git status                  # شوف إيه اللي اتغير
git add .                   # أو: git add src/pages/Login.jsx
git commit -m "feat: add login page"
```

رسالة الـ commit **لازم** تبدأ بنوع من دول، بعده نقطتين ومسافة. مثال: `feat: ...`

النوع	بيُستخدم إمتى	مثال
`feat`	ميزة جديدة	`feat: add user profile page`
`fix`	إصلاح bug	`fix: correct login redirect`
`style`	تنسيق بس من غير تغيير لوجيك	`style: format navbar file`
`refactor`	إعادة كتابة كود من غير ميزة جديدة	`refactor: split api service`
`docs`	توثيق	`docs: update readme`
`test`	اختبارات	`test: add cart tests`
`chore / build / ci / perf / revert`	صيانة، مكتبات، CI، أداء، تراجع	`chore: update dependencies`

عند كل commit، الجهاز بيفحص الكود (eslint + prettier) ويفحص شكل الرسالة. لو الـ commit اترفض، اقرا الـ error وصلّحه وأعد المحاولة. **متستخدمش** `--no-verify`.

6. رفع شغلك (Push)

قبل الرفع، اتأكد إن كل حاجة شغالة:

```
npm run lint
npm run test:run
npm run build
```

ارفع على **البرانش بتاعك**:

```
git push                          # لو البرانش متوصّل بـ GitHub
git push -u origin feature/your-task  # أول مرة ترفع البرانش ده
```

لو الـ push اترفض برسالة `rejected ... fetch first`، معناه إن فيه تحديث على البرانش ده موجود على GitHub ومش عندك. اسحبه الأول وبعدها ارفع:

```
git pull
git push
```

7. دمج شغلك في develop (Pull Request)

1. افتح `github.com/salemsalehtech/Work_Tech`. هيظهر لك زرار **Compare & pull request**، دوس عليه.
2. اتأكد إن `develop` = **base**، و**compare** = البرانش بتاعك.
3. املا الـ template: اكتب وصف للتغيير، علّم على نوع التغيير، وحط Screenshots لو فيه تغيير في الشكل.
4. دوس **Create pull request**.
5. لازم يحصل حاجتين: الـ CI يبقى أخضر ✓، و**عضو واحد على الأقل يعمل Approve**. بعدها يتعمل **Merge**.
6. بعد الـ merge، امسح البرانش من GitHub بزرار **Delete branch**، واعمل برانش جديد للـ task الجاية.
7. بعد الـ merge، ارجع للخطوة 4 عشان تسحب `develop` الجديد.

8. أوامر سريعة

عاوز تعمل إيه	الأمر
أنا على أنهي برانش؟ إيه اللي اتغير؟	`git status`
كل البرانشات المحلية واللي على GitHub	`git branch -a`
تحديث قائمة البرانشات من GitHub	`git fetch origin`
تنقل لبرانش تاني	`git checkout branch-name`
تاريخ آخر commits	`git log --oneline -10`
تلغي تعديلاتك على ملف (مش متحفظ)	`git restore path/to/file`
تشيل ملف من الـ add	`git restore --staged path/to/file`
تركن تعديلاتك مؤقتاً	`git stash`
ترجّع التعديلات المركونة	`git stash pop`
تعدّل رسالة آخر commit (قبل الـ push بس)	`git commit --amend -m "fix: ..."`

9. مشاكل شائعة وحلها

الرسالة	السبب والحل
`Repository not found / 404`	اللينك غلط، أو مسجّل دخول بحساب تاني. راجع `git remote -v` وصلّحه (القسم 2).
`Permission denied / 403`	إنت مش Collaborator. اطلب دعوة من صاحب الريبو، واقبلها من الإيميل أو من `github.com/notifications` .
`Support for password authentication was removed`	GitHub مبقاش بيقبل الباسورد. سجّل دخول من VS Code أو GitHub Desktop، أو استخدم Personal Access Token مكان الباسورد.
`rejected (fetch first / non-fast-forward)`	فيه تحديث على GitHub مش عندك. اعمل `git pull` وبعدها `git push` .
`protected branch update failed`	إنت بترفع على `develop` أو `main` . ارفع على البرانش بتاعك وافتح PR.
`subject may not be empty / type must be one of`	رسالة الـ commit مش بالشكل الصح. شوف القسم 5.
`CONFLICT (content)`	إنت وحد تاني عدّلتوا نفس السطر. صلّح الملف، وبعدها `git add .` و`git commit`.
`Missing script: "xxx"`	اسم الأمر غلط. الأوامر الصح: `dev` و `build` و `lint` و `test:run` و `format` .

الخلاصة اليومية:

```
git checkout develop && git pull origin develop
git checkout feature/your-task && git merge develop --no-edit
# ... اشتغل ...
git add . && git commit -m "feat: what you did"
git push
# افتح Pull Request على develop من GitHub
```